

INGENIERÍA DE SOFTWARE 3

TRABAJO PRÁCTICO 9

NICOLAS BERGAMI

4.1.1 - Agregar a nuestro pipeline una nueva etapa que dependa de nuestra etapa de Construcción y Pruebas y de la etapa de Construcción de img/imagenes Docker y subida a ACR realizada en el TP08

- ☐ Agregar tareas para crear un recurso Azure Container Instances que levante un contenedor con nuestra img/imagen de back utilizando un AppServicePlan en Linux

Creamos un Plan de App service de Linux en Azure Portal

[Todos los servicios](#) > [Planes de App Service](#) >

Crear plan de App Service ...

[Datos básicos](#) [Etiquetas](#) [Revisar y crear](#)

Los planes de App Service le ofrecen flexibilidad para asignar aplicaciones específicas a un conjunto de recursos concreto y para optimizar aún más el uso de los recursos de Azure. De esta forma, si quiere ahorrar gastos en el entorno de prueba, puede compartir un plan entre varias aplicaciones. [Más información](#)

Detalles del proyecto

Seleccione una suscripción para administrar los recursos implementados y los costos. Use los grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción *	Suscripción prueba
Grupo de recursos *	TPS-INGSOFT3UCC

[Crear nuevo](#)

Detalles del plan de App Service

Nombre *	MiAppPlan02
Sistema operativo *	☒ Linux ☐ Windows
Región *	East US

Plan de tarifa

El plan de tarifa de App Service determina la ubicación, las características, los costos y los recursos del proceso asociados a la aplicación. [Más información](#)

Plan de precios	Gratis F1 (Infraestructura compartida)
-----------------	---

[Explorar planes de precios](#)

Redundancia de zona

Un plan de App Service se puede implementar como un servicio con redundancia de zona en las regiones que lo admiten. Esta es una decisión que solo se toma en el momento de la implementación. Después de la implementación, no se podrá hacer que un plan de App Service tenga redundancia de zona. [Más información](#)

[Revisar y crear](#)

[< Anterior](#)

[Siguiente: Etiquetas >](#)

MiPlanApp02

Plan de Linux

Buscar Eliminar Envíenos sus comentarios

Introducción

- Registro de actividad
- Control de acceso (IAM)
- Etiquetas
- Diagnosticar y solucionar problemas
- Eventos (versión preliminar)
- Configuración
- Supervisión
- Automation
- Ayuda

Las métricas del plan App Service no se aplican a las SKU gratuitas y compartidas

Información esencial

Grupo de recursos (mover) : [TP5-JINGSOFT3UCC](#)

Estado : Listo

Ubicación : Brazil South

Suscripción (mover) : [Suscripción prueba](#)

Id. de suscripción : 191f08f0-a999-425c-900c-e404af0c0c67

Etiquetas (editar) : [Agregar etiquetas](#)

Plan de precios : [F1](#)

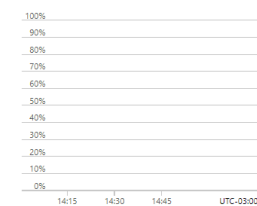
Recuento de instancias : [1](#)

Aplicaciones y espacios : [0/0](#)

Sistema operativo : Linux

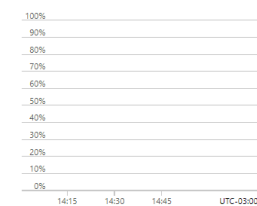
Con redundancia de zona : Deshabilitado

Porcentaje de CPU



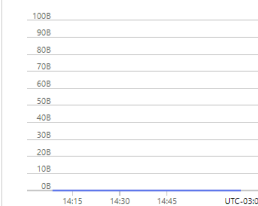
CPU Percentage (Promedio), miplanapp02 | --

Porcentaje de memoria



Memory Percentage (Promedio), miplan... | --

Datos de entrada



Data In (Suma), miplanapp02 | 0B

Datos salientes

Creamos las siguientes variables de entorno

```
38 * AppServicePlanLinux: 'MiPlanApp02'
39 * WebAppApiNameContainersQA: 'nico-crud-api-qa'
40 * WebAppFrontNameContainersQA: 'nico-crud-front-qa'
41 * baseUrlBackEndQA: 'https://$(WebAppApiNameContainersQA).azurewebsites.net'
42 * baseUrlFrontEndQA: 'https://$(WebAppFrontNameContainersQA).azurewebsites.net'
43 * WebAppApiNameContainersPROD: 'nico-crud-api-prod'
44 * WebAppFrontNameContainersPROD: 'nico-crud-front-prod'
```

y pegamos el stage de deploy app service

```
435 #####
436 #####
437 ### STAGE: DEPLOY TO AZURE APP SERVICE QA
438 #####
439 - stage: DeployImagesToAppServiceQA
440   displayName: 'Desplegar Imágenes en Azure App Service (QA)'
441   dependsOn:
442     - BuildAndTestBackAndFront
443     - DockerBuildAndPush
444   condition: succeeded()
445   jobs:
446     - job: DeployImagesToAppServiceQA
447       displayName: 'Desplegar Imágenes de API y Front en Azure App Service (QA)'
448       pool:
449         vmImage: 'ubuntu-latest'
450       steps:
451         - task: AzureCLI@2
452           displayName: 'Verificar y crear el recurso Azure App Service para API (QA) si no existe'
453           inputs:
454             azureSubscription: '$(ConnectedServiceName)'
455             scriptType: 'bash'
456             scriptLocation: 'inlineScript'
457             inlineScript: |
458               # Verificar si el App Service para la API ya existe
459               if ! az webapp list --query "[?name=='$(WebAppApiNameContainersQA)' && resourceGroup=='$(ResourceGroupName)'] | .length(@)" --no-tsv | grep
460               echo "El App Service para API QA no existe. Creando..."
461               # Crear el App Service sin especificar la imagen del contenedor
462               az webapp create --resource-group $(ResourceGroupName) --plan $(AppServicePlanLinux) --name $(WebAppApiNameContainersQA) --deployment-
463               else
464                 echo "El App Service para API QA ya existe. Actualizando la imagen..."
465               fi
466
467       # Configurar el App Service para usar Azure Container Registry (ACR)
468       az webapp config container set --name $(WebAppApiNameContainersQA) --resource-group $(ResourceGroupName) \
469       --container-image-name $(acrLoginServer)/$(backImageName):$(backImageTag) \
470       --container-registry-url https://$(acrLoginServer) \
471       --container-registry-user $(acrName) \
472       --container-registry-password $(az acr credential show --name $(acrName) --query "passwords[0].value" --no-tsv)
473
474       # Establecer variables de entorno
475       az webapp config appsettings set --name $(WebAppApiNameContainersQA) --resource-group $(ResourceGroupName) \
476       --settings ConnectionStrings_DefaultConnection="$(conn-string-qa)" \
477
```

Y vemos que los stages pasan bien y el pipeline no da errores

nicolasbergami2013 / Ultimo Trabajo / Pipelines / Ultimo Trabajo / 20241022.5

Search

+

Summary Tests Associated pipelines Code Coverage

Repository and version

Ultimo Trabajo

main 6b6903b5

Time started and elapsed

Today at 3:32 PM

9m 45s

Related

0 work items

5 published

Warnings 4

Project file(s) matching the specified pattern were not found.

Build and Test API and Front • Build and Test API • Ejecutar pruebas de la API

No code coverage results were found to publish.

Build and Test API and Front • Build and Test API • Publicar resultados de code coverage del back-end

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1729622305218.txt

Construir y Subir Imágenes Docker a ACR • Construir y Subir Imágenes Docker a ACR • Construir Imagen Docker para Back

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1729622347063.txt

Construir y Subir Imágenes Docker a ACR • Construir y Subir Imágenes Docker a ACR • Construir Imagen Docker para Front

Stages Jobs

Build and Test API an...

2 jobs completed 5m 2s

100% tests passed

4 artifacts

Construir y Subir Imá...

1 job completed 1m 38s

Desplegar Imagenes ...

1 job completed 2m 32s

Observamos que se generó la aplicación

MiPlanApp02 | Aplicaciones ☆ ...

Plan de Linux

Buscar Actualizar

Introducción

Registro de actividad

Control de acceso (IAM)

Etiquetas

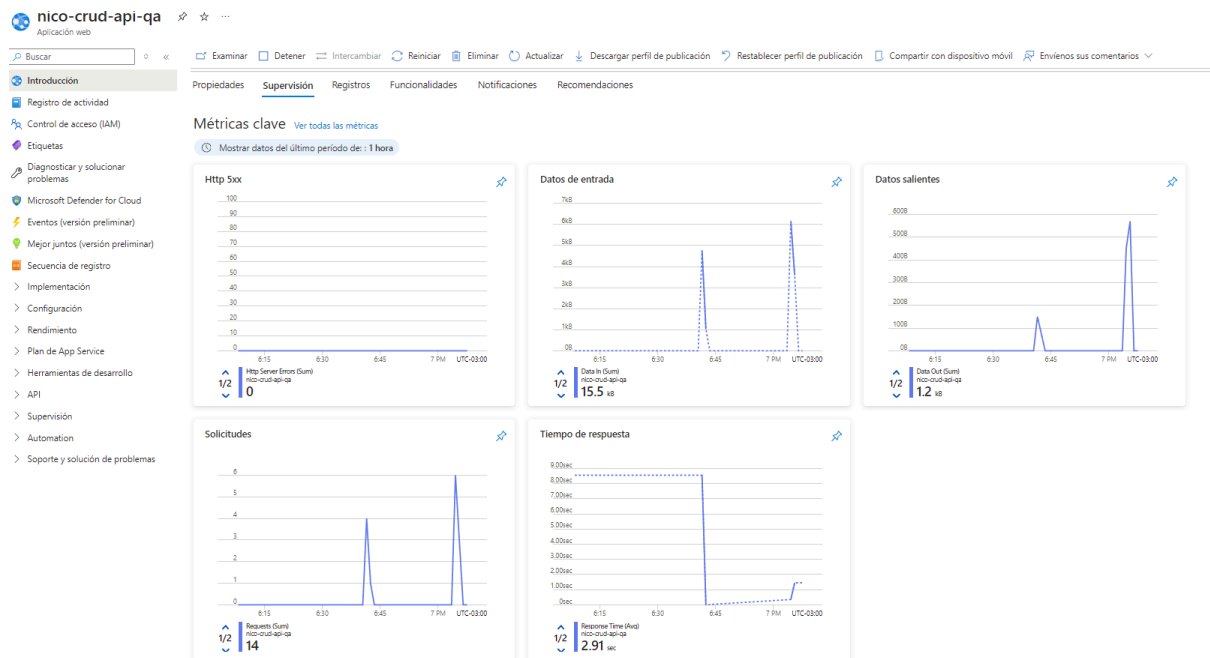
Diagnosticar y solucionar problemas

Eventos (versión preliminar)

Filtrar por nombre Estado: Todo

Nombre	Tipo	Variante	Grupo de recursos
nico-crud-api-qa	Aplicación	app.linux.container	TPS-INGSOFT3UCC

Verificamos que esté corriendo la api



```

< > ↻ 🏠 ⚠ Not secure nico-crud-api-qa.azurewebsites.net/api/Employee/GetAll
pretty-print

{
  "id": 1,
  "name": "Juan Perez",
  "createdDate": "2024-09-05T00:00:00"
},
{
  "id": 2,
  "name": "Carla Ruiz",
  "createdDate": "2024-09-05T22:22:47.440572"
},
{
  "id": 3,
  "name": "Carlos Gomez",
  "createdDate": "2024-09-06T09:16:50.070943"
},
{
  "id": 4,
  "name": "Joaquín Zarate",
  "createdDate": "2024-09-06T09:19:37.898716"
},
{
  "id": 5,
  "name": "Luis Rodriguez",
  "createdDate": "2024-09-06T09:23:31.324434"
}

```

4.2 Desafíos:

4.2.1 Agregar tareas para generar Front en Azure App Service con Soporte para Contenedores

Agregamos la siguiente variable de entorno

```
WebAppFrontNameContainersQA: 'nico-crud-front-qa'
```

Modificamos el codigo para que despliegue el front/image en Azure App Service con Soporte para Contenedores

```
312 ✓ ..... settings
313 ..... --task: AzureCLI@2
314 ..... displayName: 'Verificar y crear el recurso Azure App Service para front (QA) si no existe'
315 ..... inputs:
316 ..... azureSubscription: '$(ConnectedServiceName)'
317 ..... scriptType: 'bash'
318 ✓ ..... scriptLocation: 'inlineScript'
319 ..... inlineScript: |
320 ..... # Verificar si el App Service para la API ya existe
321 ..... if ! az webapp list --query "[?name=='$(WebAppFrontNameContainersQA)' && resourceGroup=='$(ResourceGroupName)'].length(@)>'0'" ; then
322 ..... echo "El App Service para API QA no existe. Creando..."
323 ..... # Crear el App Service sin especificar la imagen del contenedor
324 ✓ ..... az webapp create --resource-group $(ResourceGroupName) --plan $(AppServicePlanLinux) --name $(WebAppFrontNameContainersQA) --deployment
325 ..... else
326 ..... echo "El App Service para API QA ya existe. Actualizando la imagen..."
327 ..... fi
328 .....
329 ..... # Configurar el App Service para usar Azure Container Registry (ACR)
330 ..... az webapp config container set --name $(WebAppFrontNameContainersQA) --resource-group $(ResourceGroupName) \
331 ..... --container-image-name $(acrLoginServer)/$(frontImageName):$(frontImageTag) \
332 ..... --container-registry-url https://$(acrLoginServer)/ \
333 ..... --container-registry-user $(acrName) \
334 ..... --container-registry-password $(az acr credential show --name $(acrName) --query "passwords[0].value" --o tsv)
335 .....
336 .....
337 ..... # Establecer variables de entorno
338 ..... az webapp config appsettings set --name $(WebAppFrontNameContainersQA) --resource-group $(ResourceGroupName) \
339 ..... --settings API_URL="$(API_URL_QA)"
340 .....
341 .....
```

4.2.2 Agregar variables necesarias para el funcionamiento de la nueva etapa considerando que debe haber 2 entornos QA y PROD para Back y Front. Para ello vamos a tener que agregar las variables para Api de produccion y QA

Variables



 API_URL

 API_URL_PROD

 API_URL_QA
= http://nico-crud-api-qa.azurewebsites.net/api/Employee

```
· baseUrlBackEndQA: 'https://$(WebAppApiNameContainersQA).azurewebsites.net'
· baseUrlFrontEndQA: 'https://$(WebAppFrontNameContainersQA).azurewebsites.net'
```

4.2.3 Agregar tareas para correr pruebas de integración en el entorno de QA de Back y Front creado en Azure App Services con Soporte para Contenedores.

Agregamos el job de pruebas de integración para asegurarnos de que se le pase correctamente el base Url según el servicio

```
#### Integration Test with Cypress
- job: IntegrationTesting
  displayName: 'Integrations Tests'
  dependsOn: 'DeployImagesToAppServiceQA'
  variables:
    baseUrl: '$(frontContainerInstanceNameQA).azurewebsites.net' # Construir la URL
  steps:
  - script: |
    cd $(Build.SourcesDirectory)/EmployeeCrudAngular
    npm install typescript ts-node
    displayName: 'Install TypeScript'

  - # Ejecutar pruebas de Cypress
    script: |
    cd $(Build.SourcesDirectory)/EmployeeCrudAngular
    npx cypress run --config-file cypress.config.ts --env baseUrl=$(baseUrl)
    displayName: 'Run Cypress E2E Tests'

  - # Publicar los resultados de las pruebas
    Settings
    task: PublishTestResults@2
    inputs:
      testResultsFiles: '$(Build.SourcesDirectory)/EmployeeCrudAngular/cypress/results/*.xml'
      testRunTitle: 'Cypress E2E Tests (QA)'
    displayName: 'Publicar resultados de Cypress'
```

4.2.4 Agregar etapa que dependa de la etapa de Deploy en QA que genere un entorno de PROD.

Como es habitual, creamos las variables de entorno para los contenedores de Producción

```
WebAppApiNameContainersPROD: 'nico-crud-api-prod'
WebAppFrontNameContainersPROD: 'nico-crud-front-prod'
```

Y luego creamos el stage con las tareas de deploy a prod

```

#-----
#DEPLOY A PROD
#-----

- stage: DeployImagesToAppServicePROD
  displayName: 'App Service (PROD)'
  dependsOn: DeployImagesToAppServiceQA
  jobs:
  - deployment: DeployToProd
    displayName: 'Desplegar Imágenes de API y Front en Azure App Service (PROD)'
    environment: 'Production'
    strategy:
      runOnce:
        deploy:
          steps:

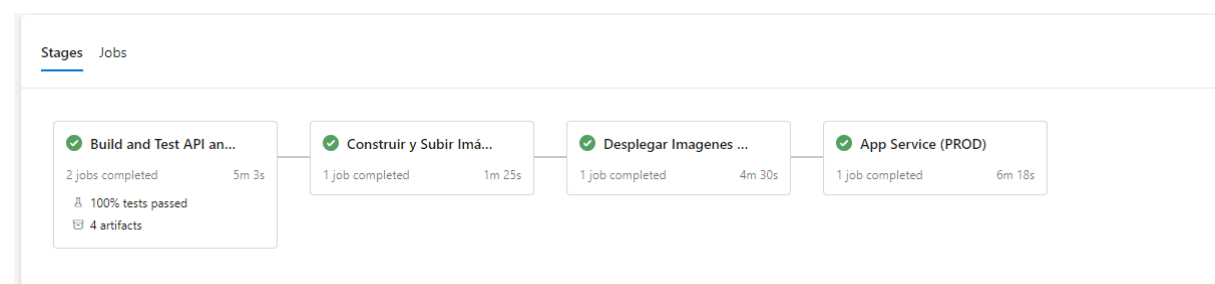
#-----
# DEPLOY DOCKER BACK IMAGE A AZURE APP SERVICE PROD
#-----

      Settings
      - task: AzureCLI@2
        displayName: 'Verificar y crear el recurso Azure App Service para API (PROD) si no existe'
        inputs:
          azureSubscription: '$(ConnectedServiceName)'
          scriptType: 'bash'
          scriptLocation: 'inlineScript'
          inlineScript: |
            # Verificar si el App Service para la API ya existe
            if ! az webapp list --query "[?name=='$(WebAppApiNameContainersPROD)' && resourceGroup=='$(ResourceGroupName)'] | length(@)" --o tsv | grep -q '^1$'; then
              echo "El App Service para API QA no existe. Creando..."
              # Crear el App Service sin especificar la imagen del contenedor
              az webapp create --resource-group $(ResourceGroupName) --plan $(AppServicePlanLinux) --name $(WebAppApiNameContainersPROD) --deployment-container-image
            else
              echo "El App Service para API QA ya existe. Actualizando la imagen..."
            fi

407         # Configurar el App Service para usar Azure Container Registry (ACR)
408         az webapp config container set --name $(WebAppApiNameContainersPROD) --resource-group $(ResourceGroupName) \
409         --container-image-name $(acrLoginServer)/$(backImageName):$(backImageTag) \
410         --container-registry-url https://$(acrLoginServer)/ \
411         --container-registry-user $(acrName) \
412         --container-registry-password $(az acr credential show --name $(acrName) --query "passwords[0].value" --o tsv)
413
414         # Establecer variables de entorno
415         az webapp config appsettings set --name $(WebAppApiNameContainersPROD) --resource-group $(ResourceGroupName) \
416         --settings ConnectionStrings__DefaultConnection="$(conn-string-prod)"
417
418
419      Settings
420      - task: AzureCLI@2
421        displayName: 'Verificar y crear el recurso Azure App Service para FRONT (PROD) si no existe'
422        inputs:
423          azureSubscription: '$(ConnectedServiceName)'
424          scriptType: 'bash'
425          scriptLocation: 'inlineScript'
426          inlineScript: |
427            # Verificar si el App Service para la API ya existe
428            if ! az webapp list --query "[?name=='$(WebAppFrontNameContainersPROD)' && resourceGroup=='$(ResourceGroupName)'] | length(@)" --o tsv | grep -q '^1$'; th
429            echo "El App Service para API QA no existe. Creando..."
430            # Crear el App Service sin especificar la imagen del contenedor
431            az webapp create --resource-group $(ResourceGroupName) --plan $(AppServicePlanLinux) --name $(WebAppFrontNameContainersPROD) --deployment-container-ima
432            else
433              echo "El App Service para API QA ya existe. Actualizando la imagen..."
434            fi
435
436            # Configurar el App Service para usar Azure Container Registry (ACR)
437            az webapp config container set --name $(WebAppFrontNameContainersPROD) --resource-group $(ResourceGroupName) \
438            --container-image-name $(acrLoginServer)/$(frontImageName):$(frontImageTag) \
439            --container-registry-url https://$(acrLoginServer)/ \
440            --container-registry-user $(acrName) \
441            --container-registry-password $(az acr credential show --name $(acrName) --query "passwords[0].value" --o tsv)
442
443            # Establecer variables de entorno
444            az webapp config appsettings set --name $(WebAppFrontNameContainersPROD) --resource-group $(ResourceGroupName) \
445            --settings API_URL="$(API_URL_PROD)"
446

```

Observamos que se ejecute correctamente y se despliegue en Prod



y se generaron los 4 App Service

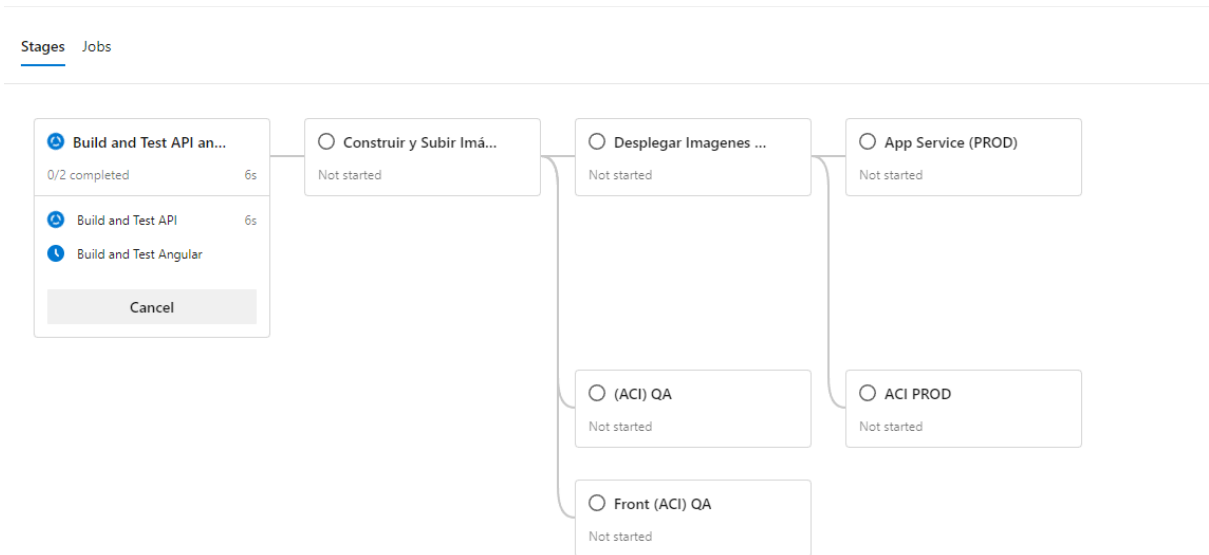
Nombre ▾	Tipo ▾	Variante ▾	Grupo de recursos ▾
nico-crud-api-prod	Aplicación	app,linux,container	tps-ingsoft3ucc
nico-crud-api-qa	Aplicación	app,linux,container	TPS-INGSOFT3UCC
nico-crud-front-prod	Aplicación	app,linux,container	tps-ingsoft3ucc
nico-crud-front-qa	Aplicación	app,linux,container	TPS-INGSOFT3UCC

Vemos si el front Corre

4.2.5 Entregar un pipeline que incluya:

- A) Etapa Construcción y Pruebas Unitarias y Code Coverage Back y Front
- B) Construcción de Imágenes Docker y subida a ACR
- C) Deploy Back y Front en QA con pruebas de integración para Azure Web Apps
- D) Deploy Back y Front en QA con pruebas de integración para ACI
- E) Deploy Back y Front en QA con pruebas de integración para Azure Web Apps con Soporte para contenedores
- F) Aprobación manual de QA para los puntos C,D,E
- G) Deploy Back y Front en PROD para Azure Web Apps
- H) Deploy Back y Front en PROD para ACI
- I) Deploy Back y Front en PROD para Azure Web Apps con Soporte para contenedores

En este punto los Stages nos quedarían de la siguiente manera



Esperemos a que todos corran correctamente

nicolasbergami2013 / Ultimo Trabajo / Pipelines / Ultimo Trabajo / 20241022.17

Summary Tests Environments Associated pipelines Code Coverage

Warnings 4

- Project file(s) matching the specified pattern were not found.
Build and Test API and Front • Build and Test API • Ejecutar pruebas de la API
- No code coverage results were found to publish.
Build and Test API and Front • Build and Test API • Publicar resultados de code coverage del back-end
- No data was written into the file /home/vsts/work/_temp/task_outputs/build_1729641375060.txt
Construir y Subir Imágenes Docker a ACR • Construir y Subir Imágenes Docker a ACR • Construir Imagen Docker para Back
- No data was written into the file /home/vsts/work/_temp/task_outputs/build_1729641401056.txt
Construir y Subir Imágenes Docker a ACR • Construir y Subir Imágenes Docker a ACR • Construir Imagen Docker para Front

Stages Jobs

```
graph LR; S1[Build and Test API an... 2 jobs completed 4m 12s 100% tests passed 4 artifacts] --> S2[Construir y Subir Imá... 1 job completed 1m 21s]; S2 --> S3[Desplegar Imágenes ... 1 job completed 5m 30s]; S3 --> S4[App Service (PROD) 1 job completed 4m 39s]; S2 --> S5[(ACI) QA 1 job completed 1m 52s]; S5 --> S6[Front (ACI) QA 1 job completed 58s]; S2 --> S7[(ACI) PROD 1 job completed 5m 32s];
```

Stages Jobs

Name	Status	Stage	Duration
✓ Build and Test API	Success	Build and Test ...	⌚ 1m 55s
✓ Build and Test Angular	Success	Build and Test ...	⌚ 1m 44s
✓ Construir y Subir Imágenes Docker a ACR	Success	Construir y Sub...	⌚ 1m 14s
✓ Desplegar en Azure Container Instances (ACI) QA	Success	(ACI) QA	⌚ 1m 46s
✓ Desplegar Front ACI QA	Success	Front (ACI) QA	⌚ 53s
✓ Desplegar Imágenes de API y Front en Azure App Service (QA)	Success	Desplegar Imag...	⌚ 5m 24s
✓ Desplegar Imágenes de API y Front en Azure App Service (PROD)	Success	App Service (PR...	⌚ 4m 34s
✓ Desplegar en Azure Container Instances PROD	Success	ACI PROD	⌚ 5m 27s

.YAML FINAL

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  solution: '**/*.sln'
  buildPlatform: 'Any CPU'
  buildConfiguration: 'Release'
  frontPath: './EmployeeCrudAngular'
  ConnectedServiceName: 'ServiceConnectionARM' # Por ejemplo
'ServiceConnectionARM'
  acrName: 'NBIngSoft3UCCACR' # Nombre del ACR sin .azurecr.io
  acrLoginServer: 'nbingsoft3uccacr.azurecr.io' # Por ejemplo
'nbingsoft3uccacr.azurecr.io'
  backImageName: 'employee-crud-api' # Nombre de la imagen del backend
  frontImageName: 'employee-crud-front'
  azureSubscription: 'ServiceConnectionARM' # El nombre exacto del Service
Connection configurado
  ResourceGroupName: 'TPS-INGSOFT3UCC' #Por ejemplo 'TPS-INGSOFT3-UCC'
  backContainerInstanceNameQA: 'nico-crud-api-qa' #Por ejemplo
'as-crud-api-qa'
  backImageTag: 'latest'
  container-cpu-api-qa: 1 #CPUS de nuestro container de QA
  container-memory-api-qa: 1.5 #RAM de nuestro container de QA
  cnn-string-qa:
'Server=tcp:as-sql-server01.database.windows.net,1433;Initial
Catalog=as-sql-db;Persist Security Info=False;User
ID=sqladmin;Password=Nico1234;MultipleActiveResultSets=False;Encrypt=True;Tr
ustServerCertificate=False;Connection Timeout=30'
  Configuration: 'Release'
  frontContainerInstanceNameQA: 'nico-crud-front-qa'
  frontImageTag: 'latest'
```

```

    container-cpu-front-qa: 1
    container-memory-front-qa: 1.5
    baseUrlBackEnd:
'http://$(backContainerInstanceNameQA).eastus.azurecontainer.io'
    baseUrlFrontEnd:
'http://$(frontContainerInstanceNameQA).eastus.azurecontainer.io'
    backContainerInstanceNamePROD: 'nico-crud-api-prod'
    frontContainerInstanceNamePROD: 'nico-crud-front-prod'
    container-cpu-api-prod: 1
    container-memory-api-prod: 1.5
    container-cpu-front-prod: 1
    container-memory-front-prod: 1.5
    cnn-string-prod:
'Server=tcp:as-sql-server01.database.windows.net,1433;Initial
Catalog=as-sql-db;Persist Security Info=False;User
ID=sqladmin;Password=Nico1234;MultipleActiveResultSets=False;Encrypt=True;Tr
ustServerCertificate=False;Connection Timeout=30'
    AppServicePlanLinux: 'MiPlanApp02'
    WebAppApiNameContainersQA: 'nico-crud-api-qa'
    WebAppFrontNameContainersQA: 'nico-crud-front-qa'
    baseUrlBackEndQA: 'https://$(WebAppApiNameContainersQA).azurewebsites.net'
    baseUrlFrontEndQA:
'https://$(WebAppFrontNameContainersQA).azurewebsites.net'
    WebAppApiNameContainersPROD: 'nico-crud-api-prod'
    WebAppFrontNameContainersPROD: 'nico-crud-front-prod'

stages:
- stage: BuildAndTest
  displayName: "Build and Test API and Front"
  jobs:
  - job: BuildDotnet
    displayName: "Build and Test API"
    pool:
      vmImage: 'windows-latest'
    steps:
    - checkout: self
      fetchDepth: 0

    - task: DotNetCoreCLI@2
      displayName: 'Restaurar paquetes NuGet'
      inputs:
        command: restore
        projects: '$(solution)'

    - task: DotNetCoreCLI@2
      displayName: 'Ejecutar pruebas de la API'
      inputs:

```

```

    command: 'test'
    projects: '/*.Tests.csproj'
    arguments: '--collect:"XPlat Code Coverage"'

- task: PublishCodeCoverageResults@2
  displayName: 'Publicar resultados de code coverage del back-end'
  inputs:
    summaryFileLocation: '$(Agent.TempDirectory)//*.cobertura.xml'
    failIfCoverageEmpty: false

- task: DotNetCoreCLI@2
  displayName: 'Compilar la API'
  inputs:
    command: build
    projects: '$(solution)'
    arguments: '--configuration $(buildConfiguration)'

- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: publish
    publishWebProjects: True
    arguments: '--configuration $(buildConfiguration) --output
$(Build.ArtifactStagingDirectory)'
    zipAfterPublish: True

- task: PublishBuildArtifacts@1
  displayName: 'Publicar artefactos de compilación'
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'api-drop'
    publishLocation: 'Container'

- job: BuildAngular
  displayName: "Build and Test Angular"
  pool:
    vmImage: 'ubuntu-latest'
  steps:
    - task: NodeTool@0
      displayName: 'Instalar Node.js'
      inputs:
        versionSpec: '22.x'

    - script: npm install
      displayName: 'Instalar dependencias'
      workingDirectory: $(frontPath)

    - script: npx ng test --karma-config=karma.conf.js --watch=false

```

```

--browsers ChromeHeadless --code-coverage
  displayName: 'Ejecutar pruebas del front'
  workingDirectory: $(frontPath)
  continueOnError: true # Continue on test failure

- task: PublishCodeCoverageResults@2
  displayName: 'Publicar resultados de code coverage del front'
  inputs:
    summaryFileLocation: '$(frontPath)/coverage/lcov.info'
    failIfCoverageEmpty: false
  condition: always()
- task: PublishTestResults@2
  displayName: 'Publicar resultados de pruebas unitarias del front'
  inputs:
    testResultsFormat: 'JUnit'
    testResultsFiles: '$(frontPath)/test-results/test-results.xml'
    failTaskOnFailedTests: true
  condition: always()

- script: npm run build
  displayName: 'Compilar el proyecto Angular'
  workingDirectory: $(frontPath)

- task: PublishBuildArtifacts@1
  displayName: 'Publicar artefactos Angular'
  inputs:
    PathToPublish: '$(frontPath)/dist'
    ArtifactName: 'front-drop'

- task: PublishPipelineArtifact@1
  displayName: 'Publicar Dockerfile de Back'
  inputs:
    targetPath: '$(Build.SourcesDirectory)/docker/api/Dockerfile'
    artifact: 'dockerfile-back'

- task: PublishPipelineArtifact@1
  displayName: 'Publicar Dockerfile de Back'
  inputs:
    targetPath: '$(Build.SourcesDirectory)/docker/front/Dockerfile'
    artifact: 'dockerfile-front'

# #-----
# ### STAGE BUILD DOCKER IMAGES Y PUSH A AZURE CONTAINER REGISTRY
# #-----

- stage: DockerBuildAndPush
  displayName: 'Construir y Subir Imágenes Docker a ACR'
  dependsOn: BuildAndTest #NOMBRE DE NUESTRA ETAPA DE BUILD Y TEST

```

```

jobs:
  - job: docker_build_and_push
    displayName: 'Construir y Subir Imágenes Docker a ACR'
    pool:
      vmImage: 'ubuntu-latest'

    steps:
      - checkout: self

      #-----
      # BUILD DOCKER BACK IMAGE Y PUSH A AZURE CONTAINER REGISTRY
      #-----

      - task: DownloadPipelineArtifact@2
        displayName: 'Descargar Artefactos de Back'
        inputs:
          buildType: 'current'
          artifactName: 'api-drop'
          targetPath: '$(Pipeline.Workspace)/drop-back'

      - task: DownloadPipelineArtifact@2
        displayName: 'Descargar Dockerfile de Back'
        inputs:
          buildType: 'current'
          artifactName: 'dockerfile-back'
          targetPath: '$(Pipeline.Workspace)/dockerfile-back'

      - task: AzureCLI@2
        displayName: 'Iniciar Sesión en Azure Container Registry (ACR)'
        inputs:
          azureSubscription: '$(ConnectedServiceName)'
          scriptType: bash
          scriptLocation: inlineScript
          inlineScript: |
            az acr login --name $(acrLoginServer)

      - task: ExtractFiles@1
        displayName: 'Descomprimir API en carpeta api'
        inputs:
          archiveFilePatterns:
            '$(Pipeline.Workspace)/drop-back/EmployeeCrudApi.zip'
          destinationFolder: '$(Pipeline.Workspace)/drop-back/api'

      - script: ls -la $(Pipeline.Workspace)/drop-back/api
        displayName: 'Verificar archivos publicados'

      - task: Docker@2
        displayName: 'Construir Imagen Docker para Back'

```

```

    inputs:
      command: build
      repository: $(acrLoginServer)/$(backImageName)
      dockerfile: $(Pipeline.Workspace)/dockerfile-back/Dockerfile
      buildContext: $(Pipeline.Workspace)/drop-back
      tags: 'latest'

- task: Docker@2
  displayName: 'Subir Imagen Docker de Back a ACR'
  inputs:
    command: push
    repository: $(acrLoginServer)/$(backImageName)
    tags: 'latest'

#-----
# BUILD DOCKER FRONT IMAGE Y PUSH A AZURE CONTAINER REGISTRY
#-----

- task: DownloadPipelineArtifact@2
  displayName: 'Descargar Artefactos de Front'
  inputs:
    buildType: 'current'
    artifactName: 'front-drop'
    targetPath: '$(Pipeline.Workspace)/drop-front'

- task: DownloadPipelineArtifact@2
  displayName: 'Descargar Dockerfile de Front'
  inputs:
    buildType: 'current'
    artifactName: 'dockerfile-front'
    targetPath: '$(Pipeline.Workspace)/dockerfile-front'

- task: AzureCLI@2
  displayName: 'Iniciar Sesión en Azure Container Registry (ACR)'
  inputs:
    azureSubscription: '$(ConnectedServiceName)'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      az acr login --name $(acrLoginServer)

- task: Docker@2
  displayName: 'Construir Imagen Docker para Front'
  inputs:
    command: build
    repository: $(acrLoginServer)/$(frontImageName)
    dockerfile: $(Pipeline.Workspace)/dockerfile-front/Dockerfile

```

```

        buildContext:
$(Pipeline.Workspace)/drop-front/employee-crud-angular/browser
        tags: 'latest'

- task: Docker@2
  displayName: 'Subir Imagen Docker de Front a ACR'
  inputs:
    command: push
    repository: $(acrLoginServer)/$(frontImageName)
    tags: 'latest'
#-----
### STAGE DEPLOY TO ACI QA
#-----

- stage: DeployToACIQA
  displayName: '(ACI) QA'
  dependsOn: DockerBuildAndPush
  jobs:
    - job: deploy_to_aci_qa
      displayName: 'Desplegar en Azure Container Instances (ACI) QA'
      pool:
        vmImage: 'ubuntu-latest'

steps:
#-----
# DEPLOY DOCKER BACK IMAGE A AZURE CONTAINER INSTANCES QA
#-----

- task: AzureCLI@2
  displayName: 'Desplegar Imagen Docker de Back en ACI QA'
  inputs:
    azureSubscription: '$(ConnectedServiceName)'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      echo "Resource Group: $(ResourceGroupName)"
      echo "Container Instance Name: $(backContainerInstanceNameQA)"
      echo "ACR Login Server: $(acrLoginServer)"
      echo "Image Name: $(backImageName)"
      echo "Image Tag: $(backImageTag)"
      echo "Connection String: $(cnn-string-qa)"

      az container delete --resource-group $(ResourceGroupName) --name
$(backContainerInstanceNameQA) --yes

      az container create --resource-group $(ResourceGroupName) \
--name $(backContainerInstanceNameQA) \

```



```

        --image $(acrLoginServer)/$(backImageName):$(backImageTag) \
        --registry-login-server $(acrLoginServer) \
        --registry-username $(acrName) \
        --registry-password $(az acr credential show --name $(acrName)
--query "passwords[0].value" -o tsv) \
        --dns-name-label $(backContainerInstanceNameQA) \
        --ports 80 \
        --environment-variables
ConnectionStrings__DefaultConnection="$(cnn-string-qa)" \
        --restart-policy Always \
        --cpu $(container-cpu-api-qa) \
        --memory $(container-memory-api-qa)

#-----
### STAGE DEPLOY TO ACI QA FOR FRONT
#-----

- stage: DeployToACIQAFront
  displayName: 'Front (ACI) QA'
  dependsOn: DockerBuildAndPush # El contenedor del front depende de que
las imágenes estén creadas
  jobs:
    - job: deploy_to_aci_qa_front
      displayName: 'Desplegar Front ACI QA'
      pool:
        vmImage: 'ubuntu-latest'

      steps:
        #-----
        # DEPLOY DOCKER FRONT IMAGE A AZURE CONTAINER INSTANCES QA
        #-----
        - task: AzureCLI@2
          displayName: 'Desplegar Imagen Docker de Front en ACI QA'
          inputs:
            azureSubscription: '$(ConnectedServiceName)'
            scriptType: bash
            scriptLocation: inlineScript
            inlineScript: |
              echo "Resource Group: $(ResourceGroupName)"
              echo "Container Instance Name: $(frontContainerInstanceNameQA)"
              echo "ACR Login Server: $(acrLoginServer)"
              echo "Image Name: $(frontImageName)"
              echo "Image Tag: $(frontImageTag)"
              echo "API URL: $(API_URL)"

```

```
    az container delete --resource-group $(ResourceGroupName) --name  
$(frontContainerInstanceNameQA) --yes
```

```
    az container create --resource-group $(ResourceGroupName) \  
--name $(frontContainerInstanceNameQA) \  
--image $(acrLoginServer)/$(frontImageName):$(frontImageTag) \  
--registry-login-server $(acrLoginServer) \  
--registry-username $(acrName) \  
--registry-password $(az acr credential show --name $(acrName)  
--query "passwords[0].value" -o tsv) \  
--dns-name-label $(frontContainerInstanceNameQA) \  
--ports 80 \  
--environment-variables API_URL=$(API_URL) \  
--restart-policy Always \  
--cpu $(container-cpu-front-qa) \  
--memory $(container-memory-front-qa)
```

```
#-----
```

```
### STAGE DEPLOY TO AZURE APP SERVICE QA
```

```
#-----
```

```
- stage: DeployImagesToAppServiceQA  
  displayName: 'Desplegar Imagenes en Azure App Service (QA)'  
  dependsOn:  
    - BuildAndTest  
    - DockerBuildAndPush  
  condition: succeeded()  
  jobs:  
    - job: DeployImagesToAppServiceQA  
      displayName: 'Desplegar Imagenes de API y Front en Azure App Service  
(QA)'  
      pool:  
        vmImage: 'ubuntu-latest'  
      steps:  
        #-----  
        # DEPLOY DOCKER API IMAGE TO AZURE APP SERVICE (QA)  
        #-----  
        - task: AzureCLI@2  
          displayName: 'Verificar y crear el recurso Azure App Service  
para API (QA) si no existe'  
          inputs:  
            azureSubscription: '$(ConnectedServiceName)'  
            scriptType: 'bash'  
            scriptLocation: 'inlineScript'  
            inlineScript: |  
              # Verificar si el App Service para la API ya existe
```

```

        if ! az webapp list --query
"[*name=='$(WebAppApiNameContainersQA)' &&
resourceGroup=='$(ResourceGroupName)'] | length(@)" -o tsv | grep -q '^1$';
then
        echo "El App Service para API QA no existe. Creando..."
        # Crear el App Service sin especificar la imagen del
contenedor
        az webapp create --resource-group $(ResourceGroupName)
--plan $(AppServicePlanLinux) --name $(WebAppApiNameContainersQA)
--deployment-container-image-name "nginx" # Especifica una imagen temporal
para permitir la creación
        else
        echo "El App Service para API QA ya existe. Actualizando
la imagen..."
        fi

        # Configurar el App Service para usar Azure Container
Registry (ACR)
        az webapp config container set --name
$(WebAppApiNameContainersQA) --resource-group $(ResourceGroupName) \
        --container-image-name
$(acrLoginServer)/$(backImageName):$(backImageTag) \
        --container-registry-url https://$(acrLoginServer) \
        --container-registry-user $(acrName) \
        --container-registry-password $(az acr credential show
--name $(acrName) --query "passwords[0].value" -o tsv)
        # Establecer variables de entorno
        az webapp config appsettings set --name
$(WebAppApiNameContainersQA) --resource-group $(ResourceGroupName) \
        --settings
ConnectionStrings__DefaultConnection="$(cnn-string-qa)" \

- task: AzureCLI@2
  displayName: 'Verificar y crear el recurso Azure App Service
para front (QA) si no existe'
  inputs:
    azureSubscription: '$(ConnectedServiceName)'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      # Verificar si el App Service para la API ya existe
      if ! az webapp list --query
"[*name=='$(WebAppFrontNameContainersQA)' &&
resourceGroup=='$(ResourceGroupName)'] | length(@) > `0`; then
        echo "El App Service para API QA no existe. Creando..."
        # Crear el App Service sin especificar la imagen del
contenedor
        az webapp create --resource-group $(ResourceGroupName)

```

```

--plan $(AppServicePlanLinux) --name $(WebAppFrontNameContainersQA)
--deployment-container-image-name "nginx"
    else
        echo "El App Service para API QA ya existe. Actualizando
la imagen..."
    fi

    # Configurar el App Service para usar Azure Container
Registry (ACR)
    az webapp config container set --name
$(WebAppFrontNameContainersQA) --resource-group $(ResourceGroupName) \
    --container-image-name
$(acrLoginServer)/$(frontImageName):$(frontImageTag) \
    --container-registry-url https://$(acrLoginServer)/ \
    --container-registry-user $(acrName) \
    --container-registry-password $(az acr credential show
--name $(acrName) --query "passwords[0].value" -o tsv)

    # Establecer variables de entorno
    az webapp config appsettings set --name
$(WebAppFrontNameContainersQA) --resource-group $(ResourceGroupName) \
    --settings API_URL="$(API_URL_QA)"

#-----
#DEPLOY A PROD
#-----

- stage: DeployImagesToAppServicePROD
  displayName: 'App Service (PROD)'
  dependsOn: DeployImagesToAppServiceQA
  jobs:
    - deployment: DeployToProd
      displayName: 'Desplegar Imágenes de API y Front en Azure App Service
(PROD)'
      environment: 'Production'
      strategy:
        runOnce:
          deploy:
            steps:

#-----
# DEPLOY DOCKER BACK IMAGE A AZURE APP SERVICE PROD
#-----

```

```

- task: AzureCLI@2
  displayName: 'Verificar y crear el recurso Azure App Service
para API (PROD) si no existe'
  inputs:
    azureSubscription: '$(ConnectedServiceName)'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      # Verificar si el App Service para la API ya existe
      if ! az webapp list --query
"[*?name=='$(WebAppApiNameContainersPROD)' &&
resourceGroup=='$(ResourceGroupName)'] | length(@)" -o tsv | grep -q '^1$';
then
        echo "El App Service para API QA no existe. Creando..."
        # Crear el App Service sin especificar la imagen del
contenedor
        az webapp create --resource-group $(ResourceGroupName)
--plan $(AppServicePlanLinux) --name $(WebAppApiNameContainersPROD)
--deployment-container-image-name "nginx"
      else
        echo "El App Service para API QA ya existe. Actualizando
la imagen..."
      fi

      # Configurar el App Service para usar Azure Container
Registry (ACR)
      az webapp config container set --name
$(WebAppApiNameContainersPROD) --resource-group $(ResourceGroupName) \
--container-image-name
$(acrLoginServer)/$(backImageName):$(backImageTag) \
--container-registry-url https://$(acrLoginServer)/ \
--container-registry-user $(acrName) \
--container-registry-password $(az acr credential show
--name $(acrName) --query "passwords[0].value" -o tsv)

      # Establecer variables de entorno
      az webapp config appsettings set --name
$(WebAppApiNameContainersPROD) --resource-group $(ResourceGroupName) \
--settings
ConnectionStrings__DefaultConnection="$(conn-string-prod)"

- task: AzureCLI@2
  displayName: 'Verificar y crear el recurso Azure App Service

```

```

para FRONT (PROD) si no existe'
    inputs:
        azureSubscription: '$(ConnectedServiceName)'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
        inlineScript: |
            # Verificar si el App Service para la API ya existe
            if ! az webapp list --query
"[%name=='$(WebAppFrontNameContainersPROD)' &&
resourceGroup=='$(ResourceGroupName)'] | length(@)" -o tsv | grep -q '^1$';
then
                echo "El App Service para API QA no existe. Creando..."
                # Crear el App Service sin especificar la imagen del
contenedor
                    az webapp create --resource-group $(ResourceGroupName)
--plan $(AppServicePlanLinux) --name $(WebAppFrontNameContainersPROD)
--deployment-container-image-name "nginx"
                else
                    echo "El App Service para API QA ya existe. Actualizando
la imagen..."
                fi

                # Configurar el App Service para usar Azure Container
Registry (ACR)
                    az webapp config container set --name
$(WebAppFrontNameContainersPROD) --resource-group $(ResourceGroupName) \
--container-image-name
$(acrLoginServer)/$(frontImageName):$(frontImageTag) \
--container-registry-url https://$(acrLoginServer)/ \
--container-registry-user $(acrName) \
--container-registry-password $(az acr credential show
--name $(acrName) --query "passwords[0].value" -o tsv)

                # Establecer variables de entorno
                    az webapp config appsettings set --name
$(WebAppFrontNameContainersPROD) --resource-group $(ResourceGroupName) \
--settings API_URL="$(API_URL_PROD)"

# #-----
# ### DEPLOY A PROD ACI
# #-----

- stage: DeployToACIPROD
  displayName: 'ACI PROD'
  dependsOn: DeployImagesToAppServiceQA

```

```

jobs:
- deployment: DeployToProd
  displayName: 'Desplegar en Azure Container Instances PROD'
  environment: 'Production'
  strategy:
    runOnce:
      deploy:
        steps:
          #-----
          # DEPLOY DOCKER BACK IMAGE A AZURE CONTAINER INSTANCES PROD
          #-----
          - task: AzureCLI@2
            displayName: 'Desplegar Imagen Docker de Back en ACI PROD'
            inputs:
              azureSubscription: '$(ConnectedServiceName)'
              scriptType: bash
              scriptLocation: inlineScript
              inlineScript: |
                echo "Resource Group: $(ResourceGroupName)"
                echo "Container Instance Name:
$(backContainerInstanceNamePROD)"
                echo "ACR Login Server: $(acrLoginServer)"
                echo "Image Name: $(backImageName)"
                echo "Image Tag: $(backImageTag)"
                echo "API URL: $(cnn-string-prod)"

                az container delete --resource-group
$(ResourceGroupName) --name $(backContainerInstanceNamePROD) --yes

                az container create --resource-group
$(ResourceGroupName) \
                --name $(backContainerInstanceNamePROD) \
                --image
$(acrLoginServer)/$(backImageName):$(backImageTag) \
                --registry-login-server $(acrLoginServer) \
                --registry-username $(acrName) \
                --registry-password $(az acr credential show --name
$(acrName) --query "passwords[0].value" -o tsv) \
                --dns-name-label $(backContainerInstanceNamePROD) \
                --ports 80 \
                --environment-variables
ConnectionStrings__DefaultConnection="$(cnn-string-prod)" \
                --restart-policy Always \
                --cpu $(container-cpu-api-qa) \
                --memory $(container-memory-api-qa)

```

```

- task: AzureCLI@2
  displayName: 'Desplegar Imagen Docker de Front en ACI PROD'
  inputs:
    azureSubscription: '$(ConnectedServiceName)'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      echo "Resource Group: $(ResourceGroupName)"
      echo "Container Instance Name:
$(frontContainerInstanceNamePROD)"
      echo "ACR Login Server: $(acrLoginServer)"
      echo "Image Name: $(frontImageName)"
      echo "Image Tag: $(frontImageTag)"
      echo "API URL: $(API_URL_PROD)"

      az container delete --resource-group
$(ResourceGroupName) --name $(frontContainerInstanceNamePROD) --yes

      az container create --resource-group
$(ResourceGroupName) \
      --name $(frontContainerInstanceNamePROD) \
      --image
$(acrLoginServer)/$(frontImageName):$(frontImageTag) \
      --registry-login-server $(acrLoginServer) \
      --registry-username $(acrName) \
      --registry-password $(az acr credential show --name
$(acrName) --query "passwords[0].value" -o tsv) \
      --dns-name-label $(frontContainerInstanceNamePROD) \
      --ports 80 \
      --environment-variables API_URL=$(API_URL_PROD) \
      --restart-policy Always \
      --cpu $(container-cpu-front-prod) \
      --memory $(container-memory-front-prod)

```

nico-crud-api-prod.azurewebsites.net/api/employee/GetAll
 pretty-print

```

[{"id":1,"name":"Juan Perez","createdAt":"2024-09-05T08:00:00"}, {"id":2,"name":"Carla Ruiz","createdAt":"2024-09-05T22:22:47.440572"}, {"id":3,"name":"Carlos Gomez","createdAt":"2024-09-06T09:16:58.070943"}, {"id":4,"name":"Joaquin Zarate","createdAt":"2024-09-06T09:19:37.890710"}, {"id":5,"name":"Luis Rodriguez","createdAt":"2024-09-06T09:23:33.324454"}]

```


ID	Name	Created Date
1	Juan Perez	05/09/2024 00:00:00
2	Carla Ruiz	05/09/2024 22:22:47
3	Carlos Gomez	06/09/2024 09:16:50